Frontend

Nuxt , Vue , No Ui

สร้างโปรเจกต์

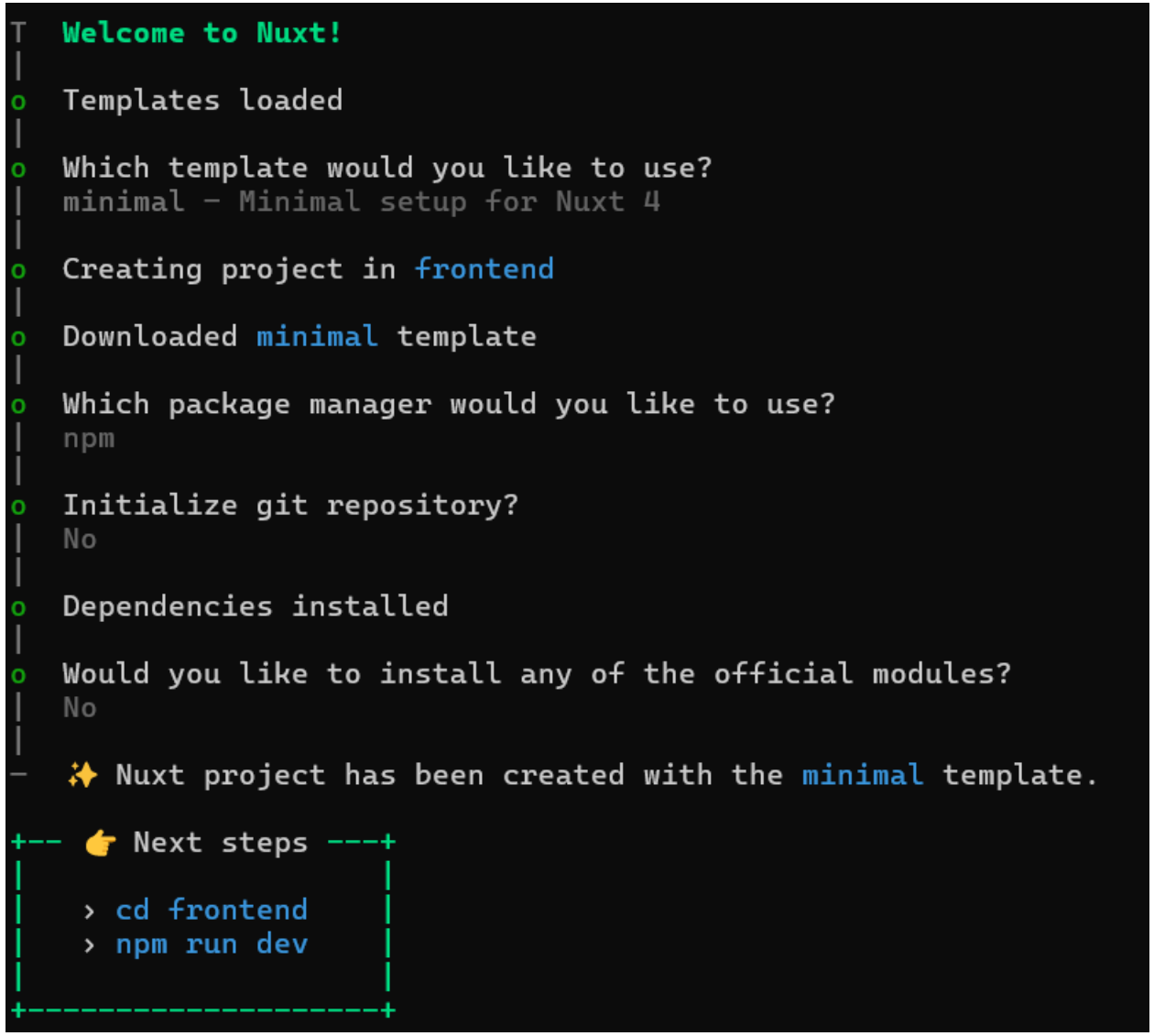
```
npx nuxi@latest init frontend
```

เข้าไปในโปรเจกต์

```
cd frontend
```

รันโปรเจกต์

```
npm run dev
```



ติดตั้ง chart.js และ vue-chartjs และ jwt-decode

```
npm install chart.js vue-chartjs jwt-decode
```

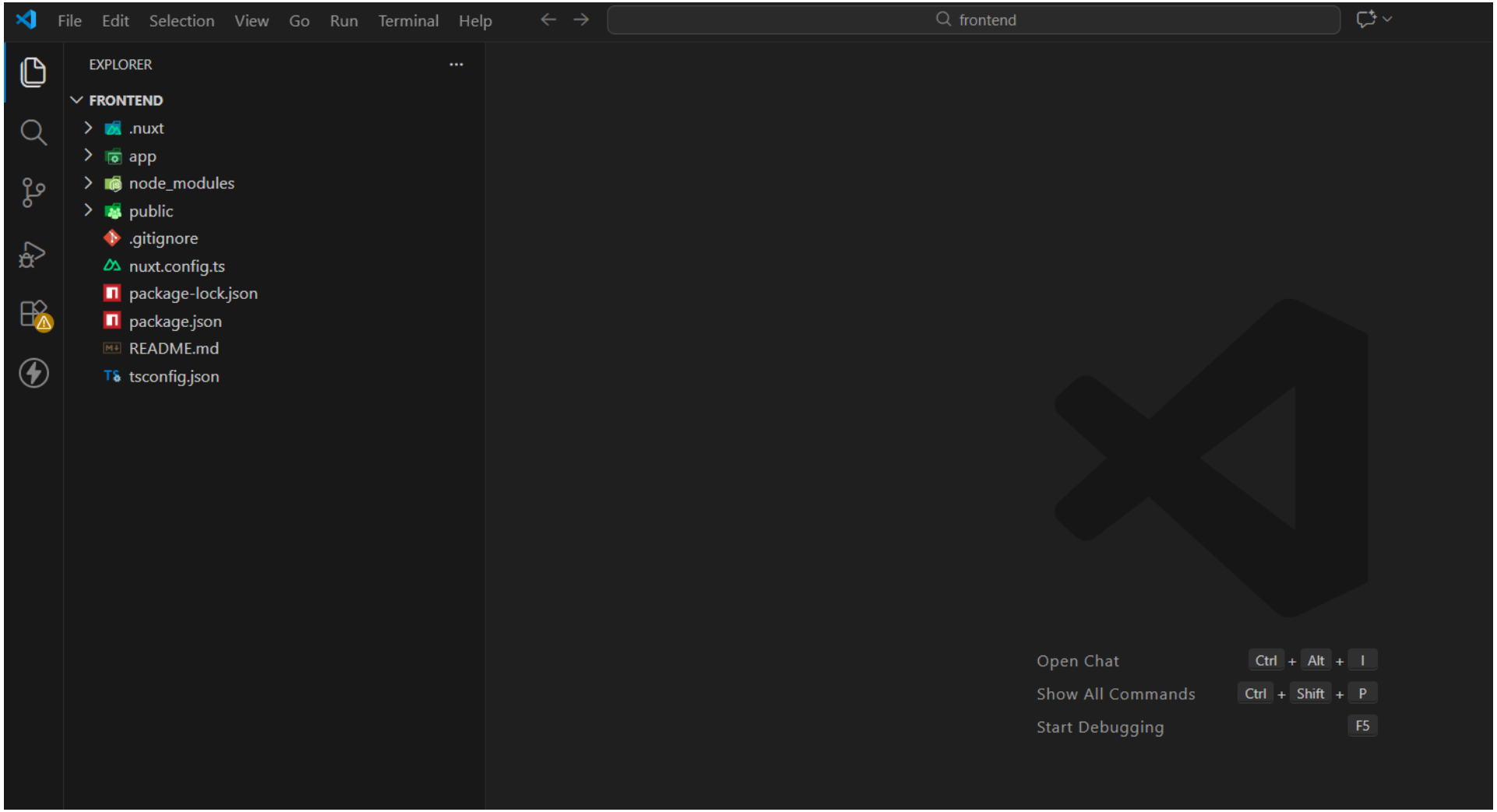
ติดตั้ง tailwindcss และกำหนดใช้งาน (หากต้องการใช้งาน)

```
npm install -D @nuxtjs/tailwindcss
```

กำหนดค่า nuxt.config.ts เพิ่ม modules: ['@nuxtjs/tailwindcss']

```
export default defineNuxtConfig({
  modules: ['@nuxtjs/tailwindcss']
})
```

เปิดโปรเจกต์ใน VS Code



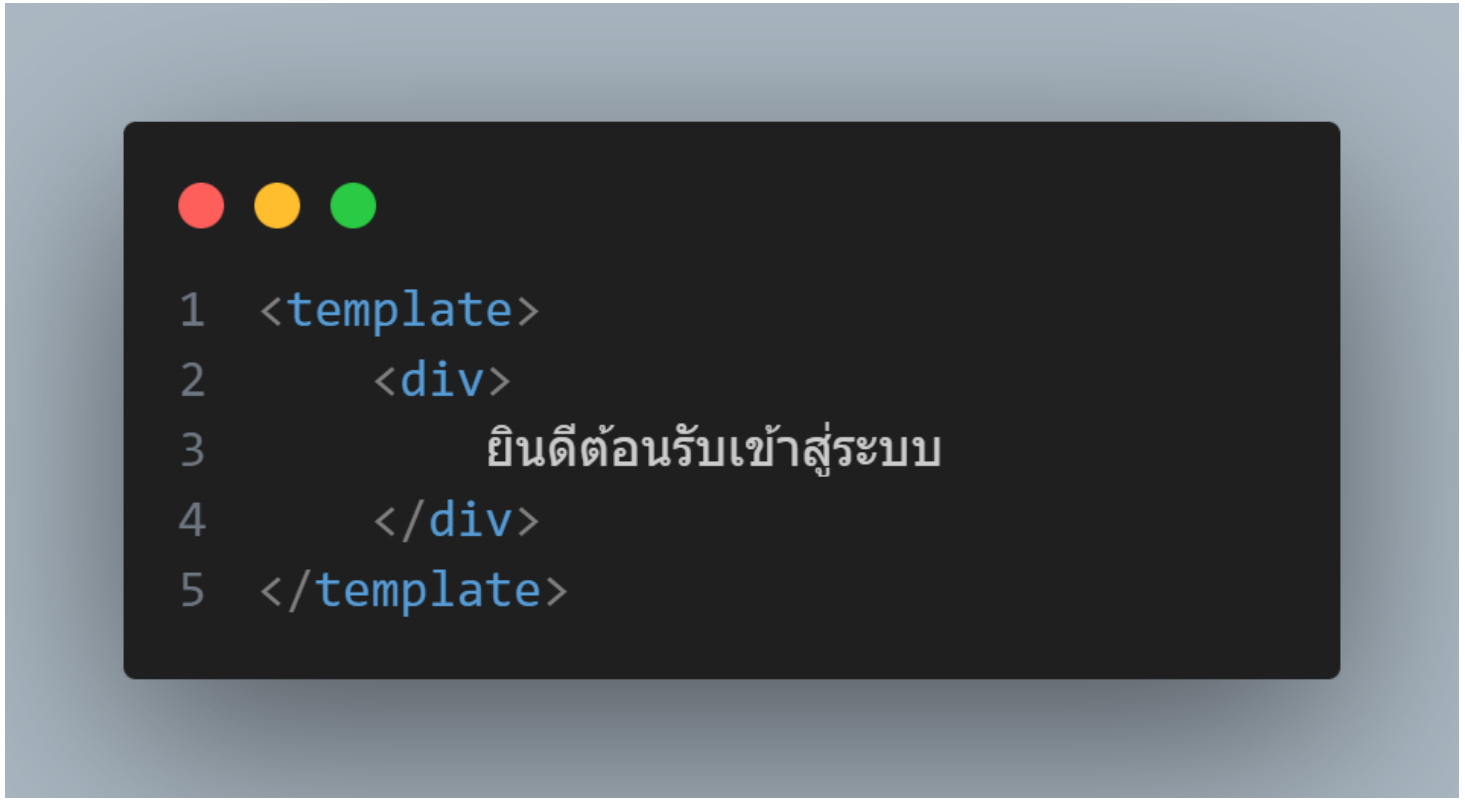
สร้างโฟนเดอร์ ไว้เก็บชิ้นงานต่างๆ ดังนี้

No	Folder	Des.	Sub Folder	File
1	composables	กำหนด function API	-	useApi.ts
2	components	กำหนดส่วนที่ใช้งานเหมือนๆ กันทุกหน้าเว็บเพจ เช่น nav , footer เป็นต้น	-	AppNavbar.vue
3	layouts	โครงสร้างหน้าเว็บ	-	default.vue empty.vue account.vue
4	middleware	กำหนดสิทธิ์การเข้าถึง	-	
5	pages	หน้าเว็บที่นำไปใช้งานใน layouts ด้วย <slot />	account	profile.vue edit-profile.vue change-photo.vue change-password.vue

Template Website

1. ในโฟลเดอร์ pages สร้างไฟล์ index.vue
2. ในโฟลเดอร์ components สร้างไฟล์ AppNavbar.vue เป็นเมนูหลัก และ AppFooter.vue เป็น Footer หลัก
3. ในโฟลเดอร์ layouts
    1. สร้างไฟล์ default.vue เป็นไฟล์โครงสร้างหลักของเว็บไซต์
    2. สร้างไฟล์ empty.vue เป็นไฟล์โครงสร้างหน้าว่างจัดกึ่งกลาง
    3. สร้างไฟล์ account.vue เป็นไฟล์โครงสร้างหน้า ตั้งค่าบัญชี
4. แก้ไขไฟล์ app.vue ให้เรียกใช้ layouts default และเรียกใช้ไฟล์ที่อยู่ใน pages
5. สร้างโฟลเดอร์ account ใน pages สำหรับจัดการ Account ของตนเอง
6. สร้าง login.vue ใน pages

1. ในโฟลเดอร์ pages สร้างไฟล์ index.vue



```
1  <template>
2      <div>
3          ยินดีต้อนรับเข้าสู่ระบบ
4      </div>
5  </template>
```

2. components สร้างไฟล์ AppNavbar.vue

1. template

```vue
<template>
  <nav class="navbar">

    <div class="group">
      <NuxtLink to="/" class="menu">หน้าแรก</NuxtLink>
      <NuxtLink to="/all" class="menu">เมนูอื่นๆ ตามระบบ</NuxtLink>
    </div>

    <div class="group">
      <span class="user">ศรัญญู (Admin)</span>
      <div class="line"></div>

      <NuxtLink to="/account/profile" class="menu">ตั้งค่าบัญชี</NuxtLink>
      <button @click="logout" class="logout">ออกจากระบบ</button>
    </div>
  </nav>
</template>
```

2. Script logout

```vue
<script setup>
    const token = useCookie('auth_token')
    const logout = () => {
        token.value = null
        navigateTo('/login')
    }
</script>
```

3. CSS nav

```vue

<style scoped>
.navbar, .group { display: flex; align-items: center; }
.navbar {
  justify-content: space-between;
  padding: 10px 20px;
  background: #fff;
  border-bottom: 1px solid #eee;
  font-family: sans-serif;
}
.group { gap: 10px; }

/* ยุบรวมเมนูและปุ่มไว้ด้วยกันในส่วนที่เหมือนกัน */
.menu, .logout {
  padding: 8px 12px;
  border-radius: 6px;
  font-size: 0.9rem;
  font-weight: bold;
  transition: 0.2s;
  text-decoration: none;
}

.menu { color: #333; }
.menu:hover, .router-link-active { background: #f0f7ff; color: #2f81ed; }

.user { font-size: 0.85rem; color: #27ae60; }
.line { width: 1px; height: 15px; background: #ddd; }

.logout {
  background: #fff0f0;
  border: 1px solid #ffccc7;
  color: #ff4d4f;
  cursor: pointer;
}
.logout:hover { background: #ff4d4f; color: #fff; }
</style>
```

2. components สร้างไฟล์ AppFooter.vue

1. template

```vue
<template>
    <div class="footer-fixed">
        @ วิทยาลัยเทคนิคกระบี่
    </div>
</template>
```

2. CSS nav

```vue
<style scoped>
.footer-fixed {
  position: fixed;
  bottom: 0;
  left: 0;
  padding: 5px 10px;
  font-size: 14px;
  color: #909090; /* สีเทาเข้ม */
}
</style>
```

3.1 ในโฟลเดอร์ layouts สร้างไฟล์ default.vue

```
1  <template>
2    <div>
3      <AppNavbar/>
4      <div style="padding: 20px;">
5              <slot />
6      </div>
7    </div>
8
9      <AppFooter/>
10 </template>
11
```

3.2 ในโฟลเดอร์ layouts สร้างไฟล์ empty.vue

```vue
<template>
  <div style="display: grid; place-items: center; min-height: 100vh;">
    <slot />
  </div>
</template>
```

3.1 ในโฟลเดอร์ layouts สร้างไฟล์ account.vue

1. template

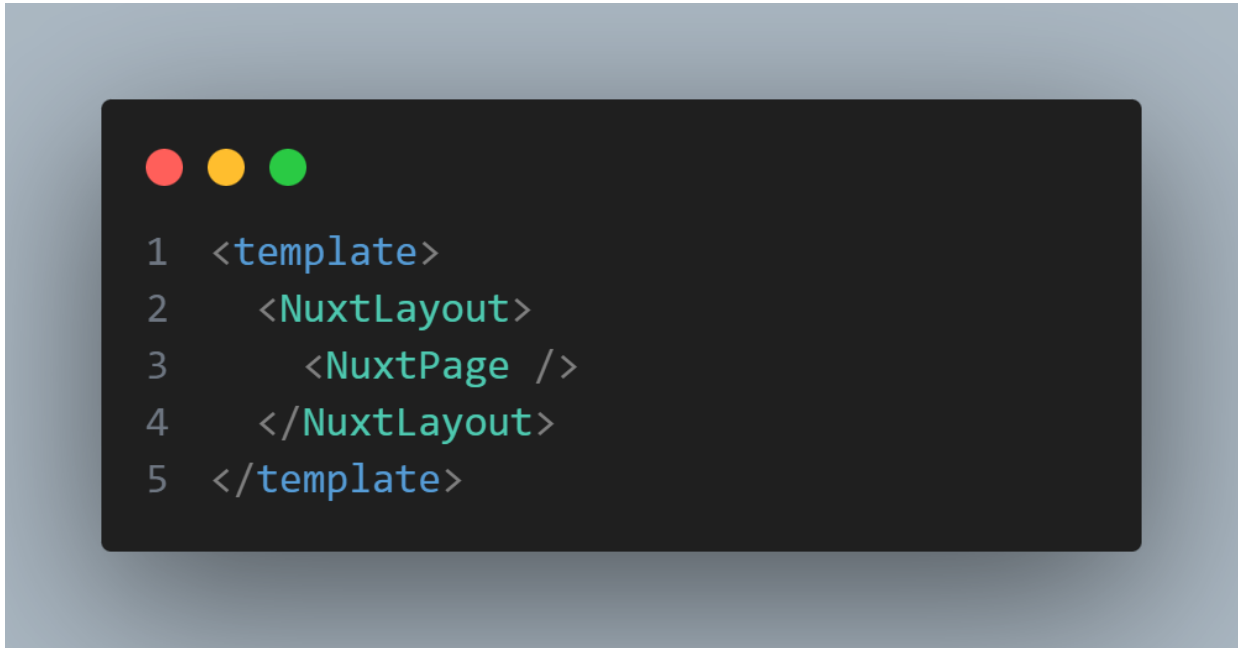
```vue
<template>
    <AppNavbar/>
  <div class="account-layout">
    <aside class="sidebar">
      <nav>
        <NuxtLink to="/account/profile" class="nav-item">ข้อมูลส่วนตัว</NuxtLink>
        <NuxtLink to="/account/change-photo" class="nav-item">เปลี่ยนรูปโปรไฟล์</NuxtLink>
        <NuxtLink to="/account/edit-profile" class="nav-item">แก้ไขข้อมูลส่วนตัว</NuxtLink>
        <NuxtLink to="/account/change-password" class="nav-item">เปลี่ยนรหัสผ่าน</NuxtLink>
      </nav>
    </aside>

    <main class="content">
      <slot /> </main>
  </div>
  <AppFooter/>
</template>
```

2. CSS nav

```vue
<style scoped>
.account-layout { display: flex; gap: 30px; padding: 20px; }
.sidebar { width: 250px; display: flex; flex-direction: column; gap: 10px; }
.nav-item { display: block; padding: 10px; text-decoration: none; color: #333; }
.router-link-active { color: #2f81ed; font-weight: bold; background: #f0f7ff; }
</style>
```

4. แก้ไขไฟล์ app.vue ให้ใช้งาน layouts ใช้งาน default อัตโนมัติ



```vue
<template>
  <NuxtLayout>
    <NuxtPage />
  </NuxtLayout>
</template>
```

ใช้ layouts กำหนดใน script หน้าที่ต้องการ definePageMeta({ layout: 'empty' })

5. สร้างโฟลเดอร์ account ใน pages และไฟล์ ดังนั้น

profile.vue	ดูข้อมูลส่วนตัว
change-photo.vue	แก้ไขรูปภาพ
edit-profile.vue	แก้ไขข้อมูลส่วนตัว
change-password.vue	แก้ไขรหัสผ่าน

template และ Script ในแต่ละหน้า

```vue
<script setup>
  definePageMeta({layout: 'account' })
</script>

<template>
  <div>
    <h1>ข้อความหัวเรื่อง</h1>
    </div>
</template>
```

6.สร้าง login.vue ใน pages

1. template

```vue
<template>
  <div class="login">
    <form @submit.prevent="login">
      <h2>เข้าสู่ระบบ</h2>
        <p v-if="error" style="color:red; font-size:14px;">
            {{ error }}
        </p>
      <div>
        <label>ชื่อผู้ใช้งาน</label>
        <input type="text" v-model="u_username" />
      </div>
      <div>
        <label>รหัสผ่าน</label>
        <input type="password" v-model="u_password" />
      </div>
      <button type="submit">เข้าสู่ระบบ</button>
    </form>
  </div>
</template>
```

2. scritp

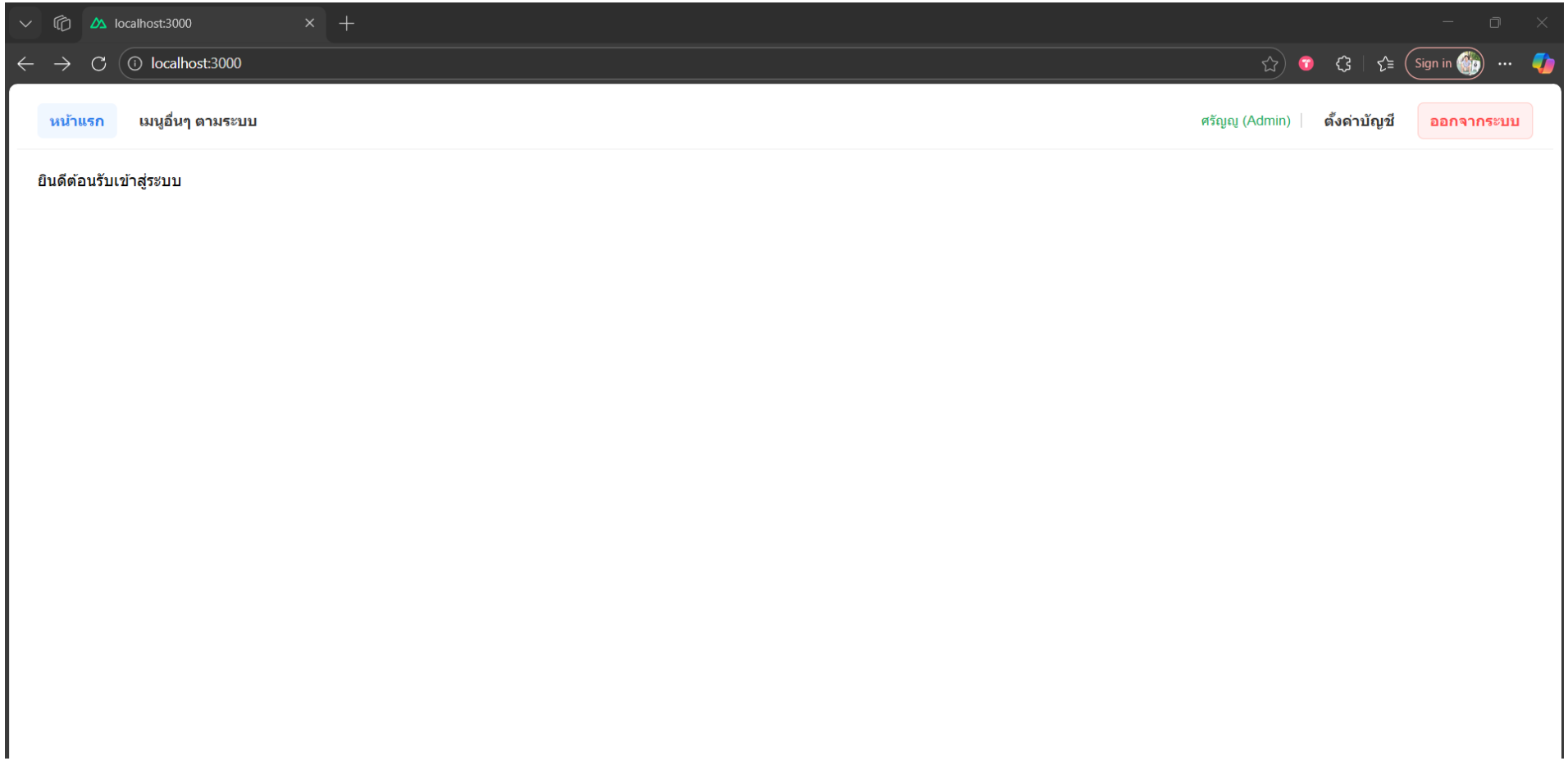
```vue
<script setup>
    definePageMeta({ layout: 'empty' })
    const error = ref('')
    const u_username = ref('')
    const u_password = ref('')

    const login = () => {
    error.value = ''

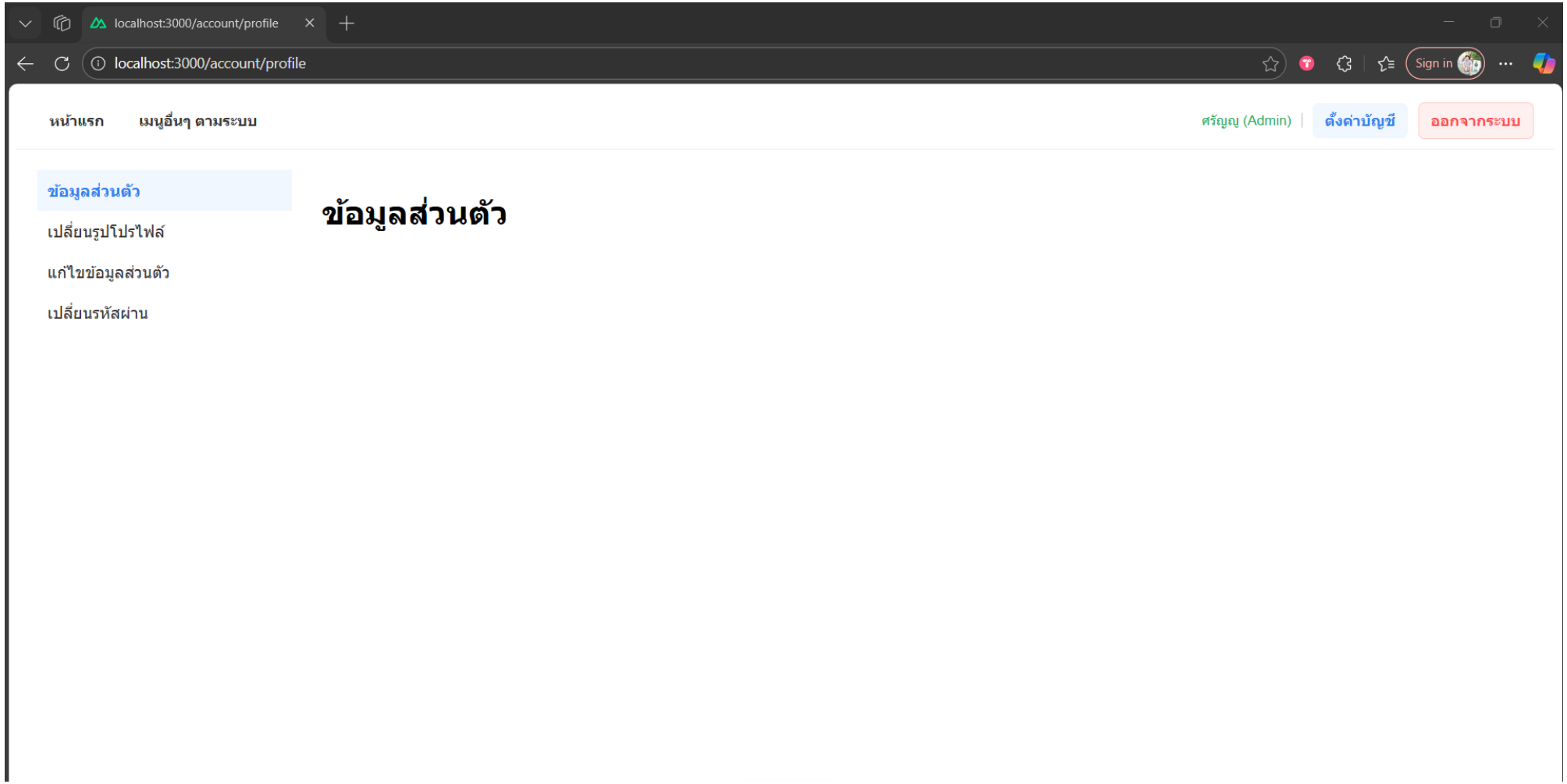
    if (!u_username.value || !u_password.value) {
        error.value = 'กรุณากรอกชื่อผู้ใช้งานและรหัสผ่าน'
        return
    }
    // ติดต่อ API ในอนาคต
    console.log(u_username.value, u_password.value)
    }
</script>
```

3. CSS ออกแบบได้เลย

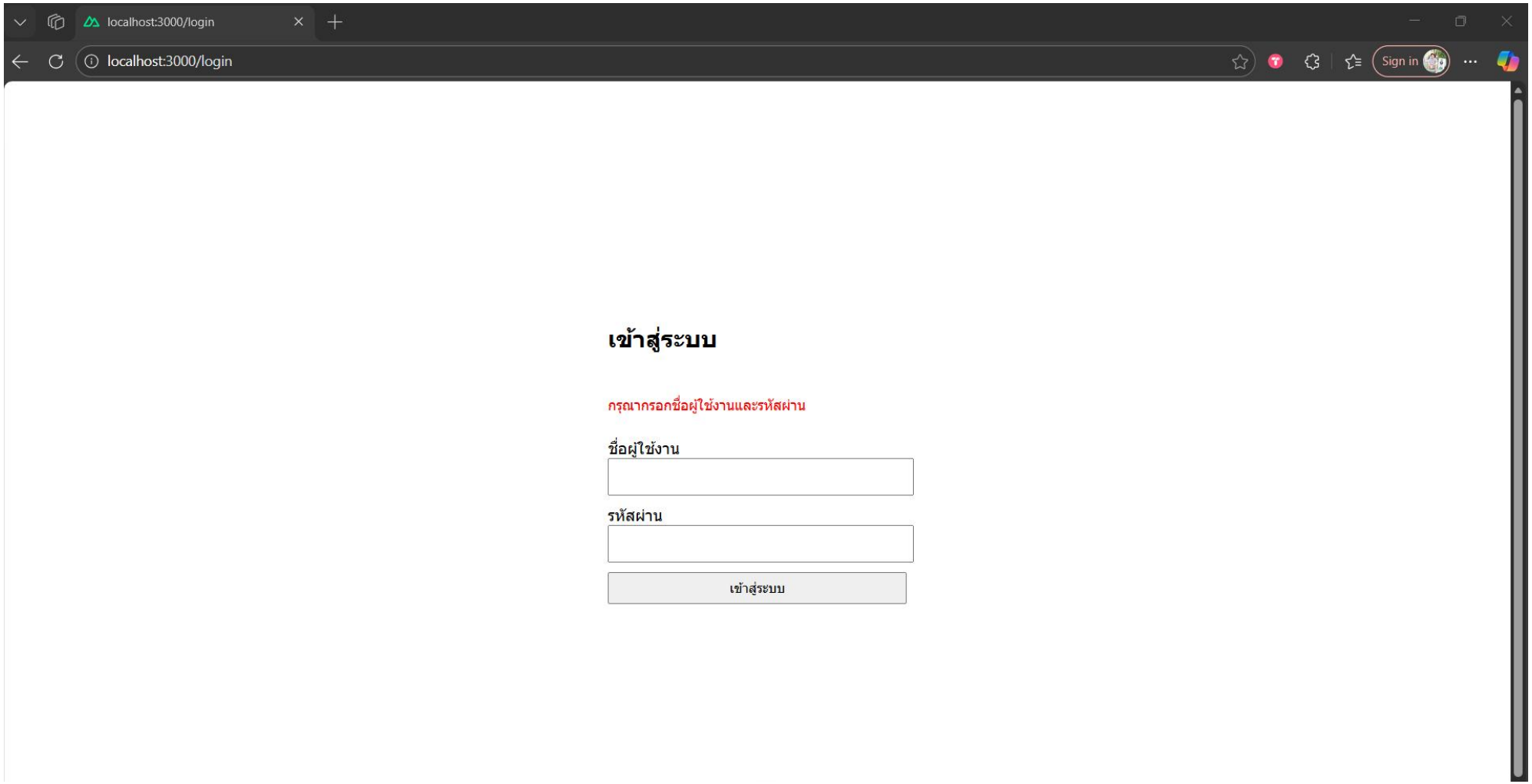
ผลลัพธ์ หน้าแรก



ผลลัพธ์ หน้าตั้งค่าบัญชี



ผลลัพธ์ เข้าสู่ระบบ



กำหนด API ให้กับ frontend

ไฟล์ nuxt.config.ts



ใส่ , ด้วยเนื่องจากเป็นข้อมูล json

เพิ่มโค้ดและกำหนด API ตามต้องการ

กำหนด API ให้กับ frontend

ไฟล์ nuxt.config.ts



ใส่ , ด้วยเนื่องจากเป็นข้อมูล json

เพิ่มโค้ดและกำหนด API ตามต้องการ

เขียน function API ไว้ใช้งาน ได้ทั้ง get post put del ใช้ Auth และไม่ใช้

1. สร้างโฟนเดอร์ composables และสร้างไฟล์ useApi.ts

   พร้อมเขียน function useApi , ประกาศตัวแปร เรียกใช้ apiBase, token



```
1  export const useApi = () => {
2    const { public: { apiBase } } = useRuntimeConfig()
3    const token = useCookie('token')
4
5
6  }
```

เขียน function API ไว้ใช้งาน ได้ทั้ง get post put del ใช้ Auth และไม่ใช้

2. เขียนการส่งค่าไปหา API ใน function useApi

```
1  const req = (method: string, url: string, body?: any) =>
2    $fetch(`${apiBase}${url}`, {
3      method,
4      body,
5      headers: token.value
6        ? { Authorization: `Bearer ${token.value}` }
7        : {}
8    })
```

เขียน function API ไว้ใช้งาน ได้ทั้ง get post put del ใช้ Auth และไม่ใช้

3. ส่งค่าไปหา Api ตาม method

```
return {
  get:  (url: string) => req('GET', url),
  post: (url: string, body: any) => req('POST', url, body),
  put:  (url: string, body: any) => req('PUT', url, body),
  del:  (url: string) => req('DELETE', url)
}
```

โค้ดเต็ม useApi.ts

```ts
export const useApi = () => {
  const { public: { apiBase } } = useRuntimeConfig()
  const token = useCookie('token')

  const req = (method: string, url: string, body?: any) =>
    $fetch(`${apiBase}${url}`, {
      method,
      body,
      headers: token.value
        ? { Authorization: `Bearer ${token.value}` }
        : {}
    })

  return {
    get:  (url: string) => req('GET', url),
    post: (url: string, body: any) => req('POST', url, body),
    put:  (url: string, body: any) => req('PUT', url, body),
    del:  (url: string) => req('DELETE', url)
  }
}

```